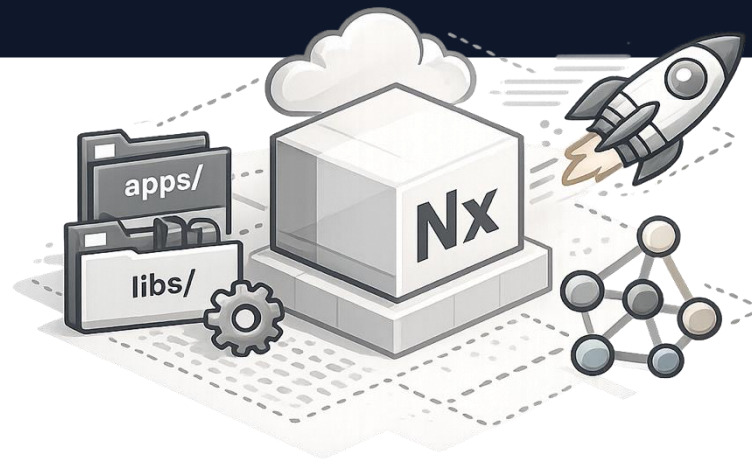




Progettazione di applicazioni web full stack

Installazione e configurazione di Nx



Prof. Devis Bianchini
Dip. di Università degli Studi di Brescia

Cos'è Nx e perché usarlo



- Build system e task runner:
 - per workspace multi-progetto (spesso *monorepo*)
 - organizza progetti
 - automatizza build/test/analisi statica del codice (*lint*)
- *Workspace*: contenitore di app e librerie
- *Monorepo*: più progetti nello stesso repo
- *Task runner*: esecuzione corrente di target (build/test/lint)
- *Project graph*: dipendenze tra progetti
- *Caching*: delle esecuzioni
- *Affected*: esegue solo i target sui file impattati dalle modifiche

Quando Nx è utile



- Nello sviluppo di progetti (repo) con più frontend/backend/librerie condivise
- Per la standardizzazione di script di build/test/lint
- Per ridurre i tempi di build/test su grandi codebase
- Per velocizzare la pipeline di **CI** (*continuous Integration*) con cache + affected
 - **affected**: riduce *quanti* task vengono lanciati
 - **cache**: riduce *quanto costano* i task che vengono lanciati



Argomento di questa lezione...

- Requisiti e compatibilità
- Creazione di workspace
- Struttura dei progetti
- Configurazione: nx.json, project.json
- Comandi base (graph, show, run)

Cosa NON faremo

- Sviluppo di app
- Architetture avanzate
- CI/CD (continuous integration/deployment) e caching remoto

Output

- Workspace creato
- Mappa dei progetti
- Config base pronta
- Checklist troubleshooting

Requisiti e compatibilità



- Node.js (preferibilmente LTS)
- Un package manager: npm/pnpm/yarn
- Una CLI e un editor
- Git (consigliato)

```
# check veloci  
node -v  
npm -v # oppure pnpm -v / yarn -v  
git --version
```



Oggetto	Descrizione
<i>Workspace</i>	Repo Nx completo
<i>Project</i>	Dentro app/lib (ha un nome e una root)
<i>Task</i>	built/test/lint...
<i>Executor</i>	Come si esegue un task
<i>Generator</i>	Crea progetti/file standard



Prima di creare il workspace

- Spazio disco sufficiente (cache può crescere)
- Permessi di scrittura nella cartella di progetto
- Il registro npm è raggiungibile (proxy/VPN aziendale)
- Node LTS installato e aggiornato
- npm/pnpm/yarn funzionante (nel caso di pnpm – store OK, niente errori EACCESS)

```
# (opzionale) pulizia se hai problemi di install
npm cache verify
# oppure: pnpm store status
npm cache clean --force
```

Creare un workspace Nx (scaffold)



Comando consigliato (usando la versione più recente dello scaffold)

```
npx create-nx-workspace@latest  
# oppure: pnpm create-nx-workspace@latest  
# oppure: yarn dlx create-nx-workspace@latest
```

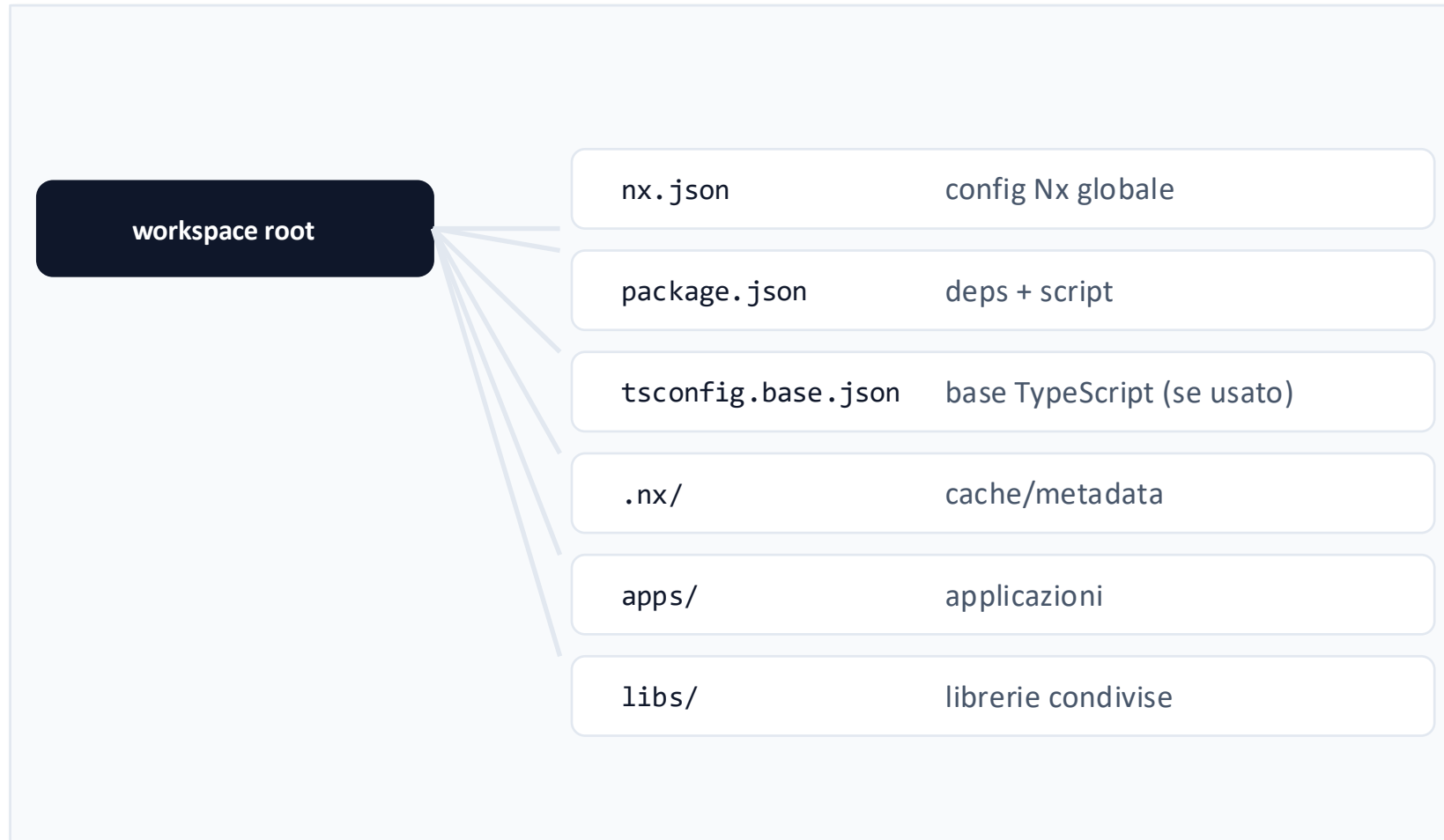
Cosa verrà chiesto in fase di installazione

- Nome workspace (cartella)
- Il «template» da cui partire (minimale, con React/Angular, customizzato)
- Se utilizzare la configurazione «full» (Nx + Nx cloud) oppure no (solo locale, senza dipendenze da cloud/servizi esterni)

Output immediato da controllare

- Cartelle create, dipendenze (file `package.json`), il *lockfile* (per rendere l'installazione riproducibile), i `node_modules`
- Il file `nx.json` (contiene regole valide per tutto il workspace, non per solo alcuni progetti)
- Il comando `'npx nx show projects'` funziona (nullo con template minimale)

Anatomia del workspace: cosa si è creato



Nx Workspace Map



nx.json: il punto di controllo del workspace



- Contiene *policies* e *comportamento di default* dell'intero workspace
 - **namedInputs** = quali file contano come input di un task (per caching)
 - **plugins** = elenco dei plugin Nx attivi (tool e file di configurazione, target/azioni disponibili, etc.)

```
{
  "$schema": "./node_modules/nx/schemas/nx-schema.json",
  "namedInputs": {
    "default": ["{projectRoot}/**/*", "sharedGlobals"],
    "production": ["default"],
    "sharedGlobals": []
  },
  "plugins": [
    {
      "plugin": "@nx/js/typescript",
      "options": {
        "typecheck": {
          "targetName": "typecheck"
        },
        "build": {
          "targetName": "build",
          "configName": "tsconfig.lib.json",
          "buildDepsName": "build-deps",
          "watchDepsName": "watch-deps"
        }
      }
    }
  ]
}
```

Configurazioni di fabbrica: plugin



Plugin (cosa sono)

- Aggiungono generator ed executor per tool specifici
- Possono inferire targets leggendo file di config (inferred tasks)
- Si scoprono con «**nx list**»

```
npx nx list  
# per vedere plugin installati e disponibili
```

Installare il plugin per NestJS (in TypeScript)



- **Nx** può ospitare un progetto **NestJS** dentro un *monorepo*, aiuta a
 - Creare app NestJS e librerie condivise
 - Organizzare codice condiviso
 - Imporre confini tra moduli/librerie
- Utile utilizzare insieme NestJS e Nx quando il repo contiene più progetti: backend NestJS, frontend React, shared libs in TypeScript

```
npx nx add @nx/nest
npx nx g @nx/nest:app apps/<app_name> # usato per creare un'applicazione NestJS
                                       # fa partire una configurazione guidata
                                       # chiede se si vogliono utilizzare eslint e jest => OK
                                       # per visualizzare tutte le opzioni

npx nx g @nx/nest:app --help
```

Avviare il backend generato con NestJS

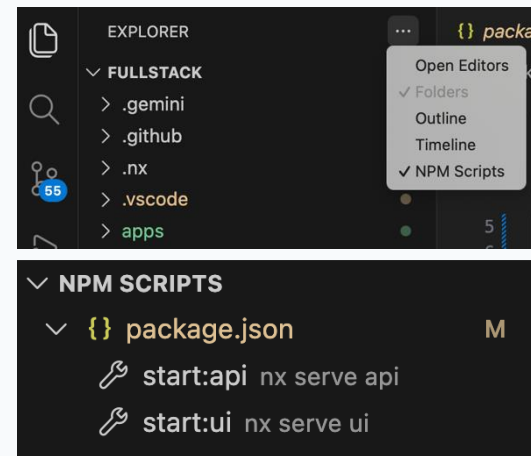


- Il comando per avviare il backend è il seguente (espone l'applicazione sulla porta :3000):

```
npx nx serve <app_name>
```

- Per comodità, si può aggiungere il comando per avviarlo direttamente dall'interfaccia di VSCode (aggiungendolo al campo `scripts` in `package.json`)

```
"scripts": {  
  "start:api": "nx serve api",  
  "start:ui": "nx serve ui"  
},
```



Config di progetto: package.json



- Ogni progetto espone dei targets (build/test/lint/...) che Nx può eseguire
 - **targets** = “cosa posso eseguire su questo progetto”
 - **executor** = implementazione del target (o run-commands)
 - **outputs** = cosa va in cache/restore
 - configurazioni per ambiente (es. prod/dev) possono vivere qui

```
"name": "@org/api",
"version": "0.0.1",
"private": true,
"nx": {
  "targets": {
    "build": {
      "executor": "nx:run-commands",
      "options": {
        "command": "webpack-cli build",
        "args": [
          "--mode=production"
        ],
        "cwd": "apps/api"
      },
      "configurations": {
        "development": {
          "args": [
            "--mode=development"
          ]
        }
      }
    }
  }
},
```

Installare il plugin per React



- Anche per **React** è necessario installare il corrispondente plugin
- Una volta installato il plugin, si procede analogamente per creare il frontend React
 - Anche in questo caso la procedura è guidata
- Il frontend si può avviare in maniera analoga
 - Come per il backend, si consiglia di impostare il comando tramite lo script in `package.json` e avviarlo dall'interfaccia di VSCode

```
npx nx add @nx/react  
  
npx nx g @nx/react:app apps/<app_name>  
  
npx nx serve <app_name>
```

```
NX Generating @nx/react:application  
✓ Which stylesheet format would you like to use? · css  
✓ Would you like to add routing to this application? (y/N) · true  
✓ Which bundler do you want to use to build the application? · vite  
✓ Which linter would you like to use? · eslint  
✓ What unit test runner should be used? · jest  
✓ Which E2E test runner would you like to use? · playwright  
✓ Which port would you like to use for the dev server? · 4200  
Fetching @nx/vite...  
Fetching @nx/playwright...
```


Configurazioni di fabbrica: generator



Generator defaults (nx.json)

- Imposta opzioni di default per generator (consistenza)
- Esempio: `npx nx g @nx/react:app`
 - Usa sempre CSS per i fogli di stile
 - Usa eslint per il controllo statico del codice
 - Usa vite per compilare e impacchettare i moduli

```
"generators": {  
  "@nx/react": {  
    "application": {  
      "babel": true,  
      "style": "css",  
      "linter": "eslint",  
      "bundler": "vite"  
    },  
    "component": {  
      "style": "css"  
    },  
    "library": {  
      "style": "css",  
      "linter": "eslint"  
    }  
  }  
}
```

Comandi per «orientarsi»



- Da eseguire in ordine (dopo lo «*scaffolding*»)

```
npx nx show projects  
npx nx show project <nome-progetto>  
npx nx graph
```

show projects

- Per capire cosa Nx considera “progetto”

show project

- Per vedere i targets disponibili, per ogni progetto
- Ottimo per debug rapido

graph

- Visualizza dipendenze
- Aiuta a capire “affected”

Eseguire task senza sviluppare (CLI)



Sintassi di base

- I target disponibili visualizzati con **«nx show project»**
- Nx gestisce dipendenze e caching se configurato

```
npx nx run <progetto>:<target>

# esempi (dipendono dai target presenti)
npx nx run my-app:lint
npx nx run my-app:test
```

Comandi multi-progetto

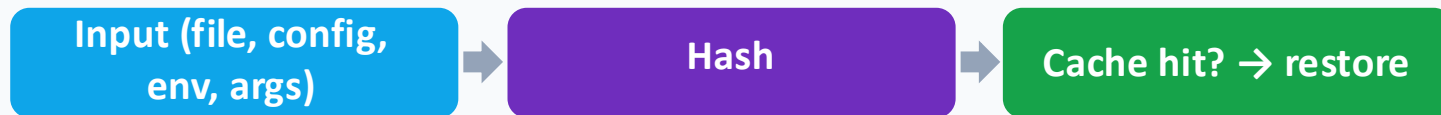
- **«run many»**: esegue un target su più progetti
- **«affected»**: si limita ai progetti impattati dalle modifiche

```
npx nx run-many -t <target>
npx nx affected -t <target>
```

Caching: cosa succede rilanciando un task



- Idea: Nx calcola un *“computation hash”* dai tuoi input → se coincide, ripristina output e log



Quando bypassare la cache

```
npx nx <target> --skip-nx-cache
```

Reset (cache + metadata)

```
npx nx reset
```

- Usalo se il project graph “impazzisce” o dopo grosse modifiche di config
- Una mossa comune di troubleshooting



- Se qualcosa non funziona...
 - 1) Verifica Node (LTS e compatibilità Nx)
 - 2) Riprova: `«npx nx reset»` (cache/metadata/daemon)
 - 3) Reinstall deps (`rm -rf node_modules` + `npm install`)
 - 4) Ricontrolla project.json / naming Progetto
 - 5) Riprova: `«npx nx show projects»` e `«npx nx graph»`

Takeaway

Un workspace Nx “capito” vale più di 10 generatori usati a caso

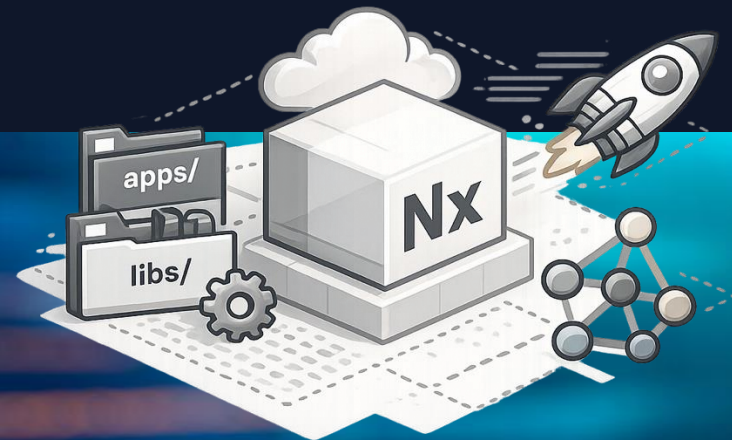


- Link alla documentazione Nx:
<https://nx.dev/docs/getting-started/installation>



Progettazione di applicazioni web full stack

Installazione e configurazione di Nx



```
mirror_mod.use_x = False
mirror_mod.use_y = True
mirror_mod.use_z = False
operation = "MIRROR_Z"
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

#selection at the end -add
obj.select= 1
obj.select=1
context.scene.objects.active
("Selected" + str(modifier))
mirror_ob.select = 0
bpy.context.selected_objects
data.objects[one.name].select
print("please select exactly

-- OPERATOR CLASSES --

types.Operator):
X_mirror to the selected
mirror_x"
```

Prof. Devis Bianchini

Dip. di Università degli Studi di Brescia